Testing and Documentation

Distributed Order Fulfilment and Reconciliation Platform · Spring Boot 3.4 (Java 17) · React + Vite · PostgreSQL · Apache Kafka

---

1. Automated tests

Run result with PostgreSQL and Kafka available: **93 tests, 0 failures, 0 errors, 0 skipped**. Without a database password, the PostgreSQL/Kafka tests are reported as skipped, never as passed.

93	0	0	27
tests run	failures / errors	skipped (with live services)	test classes

1.1 Required scenarios and the tests that cover them

Scenario	What is proven	Tests

Concurrent inventory reservations	100 competing requests against 10 units give exactly 10 successes and 90 rejections; stock never negative; no orphan reservations; every request outcome is recorded. Also verified on real PostgreSQL through the HTTP order API.	`InventoryReservationEngineTest` (H2)
		`PostgresInventoryConcurrencyIntegrationTest`
		`PostgresOrderConcurrencyIntegrationTest`

Duplicate events	A redelivered event is detected through `processed_events`, its business effect is applied once, and the duplicate is audited. Duplicate delivery through real Kafka is absorbed.	`ConsumerRetryAndDeadLetterTest`
		`EventProcessingTest`
		`FailedEventWorkflowTest`

Retries and dead-letter	A transient failure is redelivered and recovers; a persistent failure is dead-lettered after 3 attempts and stays unprocessed; an unparseable message is dead-lettered immediately.	`ConsumerRetryAndDeadLetterTest` `KafkaAutomaticPipelineIntegrationTest` (real Kafka)

Payment failures and duplicate callbacks	FAILED leaves the order pending; TIMED_OUT is refined by a late callback; a duplicate callback id changes nothing and is audited; a late success on a cancelled order does not resurrect it.	`PaymentProcessingTest`
		`PaymentApiTest`
		`PaymentReconciliationFlowTest`

Order cancellation and release	Cancelling (also twice) releases each reservation exactly once, for single and multi-product orders; automatic expiry releases stock too.	`OrderRequestOutcomeTest`
		`OrderLifecycleTest`
		`OrderApiTest`

All-or-nothing multi-product order	One short line rejects the whole order and reserves nothing.	`OrderRequestOutcomeTest`
Outbox publication	Without Kafka, events stay pending; a failed send records attempt and error; an acknowledged send is stamped `KAFKA`; the scheduler publishes with no manual call.	`OutboxPublisherUnitTest` `KafkaAutomaticPipelineIntegrationTest` `KafkaOutboxEndToEndIntegrationTest`
Reconciliation	Detects payment on cancelled order, missing payment, reservation inconsistency, stale items, double charge, wrong amount; re-runs do not duplicate records; corrective actions are audited.	`ReconciliationTest` `ReconciliationApiTest` `PaymentReconciliationFlowTest`

1.2 Running the tests

```
# Hermetic suite (H2, no external services)
cd backend
mvn test

# Full suite including PostgreSQL and Kafka tests (prompts for the DB password)
powershell -ExecutionPolicy Bypass -File .\scripts\run-integration-tests.ps1

# One class
powershell -ExecutionPolicy Bypass -File .\scripts\run-integration-tests.ps1 -Test
PostgresOrderConcurrencyIntegrationTest

# Service-level reservation stress test with configurable size
mvn "-
Dtest=PostgresInventoryConcurrencyIntegrationTest#testConcurrentReservationsAgainstLimitedStockOnPostgres"
"-Ddemo.stock=10" "-Ddemo.requests=50" test
```

1.3 Demonstrations against the running application

```
node scripts\concurrency-demo.mjs        # 50 concurrent POST /api/orders vs 10 units
node scripts\e2e-demo.mjs                # full workflow, 16 checks
```

Recorded result of the concurrency demo: **10 orders created (HTTP 201), 40 rejected for insufficient stock (HTTP 409), 0 available / 10 reserved, 5 retried idempotency keys returned their original orders, no negative stock**. The end-to-end script passed all 16 checks (automatic Kafka publish and consume, payment failure and success, duplicate callbacks, fulfilment, cancellation release, late payment reconciliation and refund, retry recovery, dead-lettering, duplicate event delivery).

2. Architecture and service boundaries

A modular monolith. PostgreSQL is the source of truth; Kafka carries asynchronous order and payment events. Each module owns its tables and exposes services to the others; the dashboard module only reads.

React dashboard :5173 → REST API :8080 → Module service → PostgreSQL (state + outbox, one transaction) → Outbox publisher (every 2 s) → Kafka topic → Idempotent consumer

Module	Responsibility	Owns tables

`catalog`	Products and warehouses.	products, warehouses
`inventory`	Stock per product and warehouse (available, reserved, fulfilled); reserve, release, fulfil.	inventory, reservations
`orders`	Idempotent order creation, lifecycle, request-outcome history, automatic expiry.	orders, order_items, order_requests
`payments`	Simulated payments (succeeded, failed, timed out), callbacks with deduplication, simulated refunds.	payments, payment_callbacks
`events`	Transactional outbox, Kafka publisher, idempotent consumer, retries, dead-letter, failure simulation.	outbox_events, processed_events, event_failures
`reconciliation`	Compares payments with orders and reservations; mismatch records and corrective actions.	reconciliation_records
`dashboard`	Read-only order details and audit endpoints for the UI.	(reads only)
`common`	Audit log, error mapping, correlation ids, scheduling.	audit_log

Key guarantees

Concern	Mechanism
No overselling	Rows locked in deterministic order, plus conditional update `WHERE available_qty >= :qty`, plus a database check constraint.
No partial multi-product reservation	All lines reserved in one transaction; any shortfall rolls everything back.
No duplicate orders	`Idempotency-Key` header with a unique constraint: same payload returns the original order, a different payload returns 409.
No lost events	Outbox row is written in the same transaction as the business change and marked published only after Kafka acknowledges.
No duplicate effects	Consumer effect and the `processed_events` marker commit together.
Bounded retries	Failures recorded with attempts, error and timestamps; redelivered after 2 s; dead-lettered after 3 attempts.
Consistent lifecycle	Transitions guarded under row locks; a cancelled order can never be confirmed or fulfilled by a late event or payment.

Order lifecycle

```
PENDING_PAYMENT --payment succeeded / confirm--> CONFIRMED --fulfil--> FULFILLED
       |
       +--cancel-------------------> CANCELLED   (reservations released)
       +--expire (30 min, automatic)-> CANCELLED   (reservations released)
```

Rejected purchase attempts. A request rejected for insufficient stock does not become an order. Every request outcome (accepted, rejected, replay, key conflict) is stored in `order_requests` and shown on the Order Requests page, so 50 attempts against 10 units appear as 10 orders and 50 recorded requests.

3. Database schema

Managed by Flyway migrations V1 to V7 in `backend/src/main/resources/db/migration`.

Table	Key columns and constraints
products	id, sku (unique), name, description, active, timestamps
warehouses	id, code (unique), name, location, active, timestamps
inventory	PK (product_id, warehouse_id); available_qty, reserved_qty, fulfilled_qty, each with CHECK >= 0; version
reservations	id, order_id, product_id, warehouse_id, quantity (> 0), status RESERVED / CONFIRMED / RELEASED / FULFILLED, released_at, fulfilled_at
orders	id, status, idempotency_key (unique), request_hash, currency, total_amount, confirmed_at, cancelled_at, fulfilled_at
order_items	id, order_id, product_id, quantity (> 0), unit_price, line_total
order_requests	id, idempotency_key, outcome, order_id (nullable), reason, item_count, total_quantity, created_at
payments	id, order_id, provider_reference (unique), status, amount, currency, last_callback_id, succeeded_at
payment_callbacks	id, provider_callback_id (unique), payment_id, result, received_payload, received_at
outbox_events	id, aggregate_type, aggregate_id, event_type, payload, created_at, published_at, published_via (KAFKA / SIMULATED), attempt_count, last_error
processed_events	PK (consumer_name, event_id), processed_at
event_failures	id, event_id, topic, partition, offset, consumer_name, attempt_count, last_error, status RETRYING / DEAD_LETTERED / REPLAYED / RESOLVED, payload
reconciliation_records	id, mismatch_key (unique), order_id, payment_id, mismatch_type, status, details, resolution_action, resolution_notes
audit_log	id, entity_type, entity_id, action, actor, details, created_at

4. API reference

Base URL `http://localhost:8080`. Errors return `{timestamp, status, error, message, path, correlationId, details}`.

Method	Path	Purpose
POST	/api/products, /api/warehouses	Create catalog records
POST	/api/inventory/initialize	Create stock row with initial quantity
POST	/api/inventory/adjustments	Add or remove available stock
GET	/api/inventory	Available, reserved, fulfilled per product and warehouse
POST	/api/orders	Create order (requires `Idempotency-Key`); 201, or 409 on insufficient stock
GET	/api/orders?status=	List orders
GET	/api/orders/{id}/details	Products, reservations, history, payments, callbacks, events, reconciliation, audit

POST	/api/orders/{id}/cancel | confirm | fulfil | expire	Lifecycle transitions
GET	/api/orders/requests, /requests/summary	Request outcome history and counts (filter `keyPrefix`, `outcome`)
POST	/api/orders/{id}/payments/simulate	Outcome SUCCEEDED, FAILED or TIMED_OUT
POST	/api/payments/callbacks	Provider callback, deduplicated by `callbackId`
GET	/api/payments, /api/payments/callbacks	Payment attempts and callback audit
POST	/api/reconciliation/run	Run reconciliation
GET	/api/reconciliation/records	Mismatch records
POST	/api/reconciliation/records/{id}/resolve	Action REFUND_PAYMENT, EXPIRE_ORDER, RESOLVE or DISMISS
GET	/api/events/summary, /api/events/outbox	Pipeline counters; events with publication and consumer state
POST	/api/events/outbox/{id}/redeliver	Send an event again (duplicate-delivery demo)
POST	/api/events/fault-injection	Make the next N consumptions fail (0 disarms)
GET/POST	/api/events/failures, /failures/{id}/retry	Retrying and dead-lettered events; manual retry
GET	/api/audit	Audit trail (filter by entity type and action)

Request and response examples

```
POST /api/orders          Idempotency-Key: ord-001
{ "currency": "USD",
  "items": [ {"productId": "<P1>", "quantity": 2, "unitPrice": 10.00},
             {"productId": "<P2>", "quantity": 1, "unitPrice": 5.00} ] }

201 { "id": "...", "status": "PENDING_PAYMENT", "totalAmount": 25.00,
      "items": [...], "reservations": [ {"status": "RESERVED", "quantity": 2, ...}, ... ] }

409 { "status": 409, "message": "Insufficient stock for product ...", ... }
```

```
POST /api/orders/{id}/payments/simulate        { "outcome": "TIMED_OUT" }
201 { "id": "...", "status": "TIMED_OUT", "providerReference": "sim_ref_...", "amount": 25.00 }

POST /api/payments/callbacks
{ "callbackId": "cb-123", "paymentId": "<PAYMENT>", "result": "SUCCEEDED" }
200 { "status": "SUCCEEDED", ... }       (sending the same callbackId again changes nothing)

POST /api/reconciliation/records/{id}/resolve
{ "action": "REFUND_PAYMENT", "notes": "customer refunded" }
200 { "status": "RESOLVED", "resolutionAction": "REFUND_PAYMENT", ... }
```

5. Event flow and failure handling

1. A business change (order created, confirmed, cancelled, fulfilled, expired; payment succeeded, failed, timed out, refunded) writes an `outbox_events` row in the **same database transaction**.
2. The scheduled publisher reads pending rows every 2 seconds and sends them to Kafka: orders to `order-events`, payments to `payment-events`, keyed by aggregate id.
3. Only after Kafka acknowledges is the row stamped `published_at` and `published_via = KAFKA`. A failed or timed-out send increments `attempt_count`, stores `last_error` and is retried on the next poll.

4. The consumer checks `processed_events`. A duplicate is skipped and audited. Otherwise it applies its effect and writes the marker in one transaction.
5. If processing fails, the attempt is recorded in `event_failures` and Kafka redelivers after 2 seconds. A later success marks the failure recovered.
6. After 3 failed attempts the event is marked `DEAD_LETTERED`, forwarded to `<topic>.DLT`, and the offset is committed so the partition keeps moving. It can be retried manually from the dashboard or API.

If the application crashes after Kafka acknowledged but before `published_at` is written, the event is published again on restart. Delivery is therefore at-least-once, and the idempotent consumer makes the repeat harmless.

6. Sample data

No fixed seed is needed. The demo scripts create their own isolated, timestamped data through the API:

Script	Data created
`concurrency-demo.mjs`	1 product `DEMO-CONC-<timestamp>`, 1 warehouse, stock 10; 50 purchase attempts with keys `demo-<timestamp>-req-000 ... 049`
`e2e-demo.mjs`	2 products `E2E-A / E2E-B`, 1 warehouse, stock 10 each; 5 orders covering payment, cancellation, late payment, retry and dead-letter cases (keys `e2e-<timestamp>-1 ... 5`)

To create data by hand, use the dashboard (Products, Warehouses, Inventory, then Create Order) or the API examples in section 4.

7. Running locally (Windows)

Prerequisites

- Java 17, Maven 3.9+, Node.js 18+
- PostgreSQL 17 with an empty database named `fulfilment` (Flyway creates the tables)
- Kafka 4.x in KRaft mode, for example `C:\kafka\kafka_2.13-4.3.1` with `log.dirs=C:/kafka/data`

Environment variables

Variable	Default	Note
DATABASE_URL	jdbc:postgresql://localhost:5432/fulfilment	
DATABASE_USERNAME	postgres	
DATABASE_PASSWORD	(none)	Entered at a prompt; never stored
KAFKA_BOOTSTRAP_SERVERS	localhost:9092	
RESERVATION_EXPIRY_MINUTES	30	Pending orders older than this expire automatically
EVENT_MAX_RETRIES	3	Attempts before dead-lettering

Start, in three PowerShell windows

```
# 1. Kafka
C:\kafka\kafka_2.13-4.3.1\bin\windows\kafka-server-start.bat C:\kafka\kafka_2.13-
4.3.1\config\server.properties

# 2. Backend (prompts for the PostgreSQL password; checks port 8080 first)
cd C:\Users\hp\Desktop\Assignment
powershell -ExecutionPolicy Bypass -File .\scripts\start-backend.ps1

# 3. Frontend  (npm.cmd avoids the PowerShell execution-policy block)
cd C:\Users\hp\Desktop\Assignment\frontend
npm.cmd install
npm.cmd run dev
```

Open `http://localhost:5173`. The backend is healthy when `http://localhost:8080/actuator/health` returns `UP`.

Dashboard pages

Page	Content
Orders	Orders grouped by status with counts; open one for products, reservations, status history, payments, callbacks, Kafka events, reconciliation and audit; payment, cancel, fulfil and callback actions.
Order Requests	Every creation request and its outcome, including rejections.
Inventory	Available, reserved and fulfilled per product and warehouse.
Payments, Reconciliation	Payment attempts, callbacks, mismatches, corrective actions and audit trail.
Event Pipeline, Failed Events / DLT	Publication and consumption state, retry counts, errors, dead-lettered events, failure simulation.

8. Known limitations

- Designed for a single backend instance; several instances would need row-level claiming in the outbox poller.
- The consumer records an audited processing step; a real downstream worker would plug into the same idempotent consumer.
- Unit prices come from the order request, as products have no price list.
- Failure simulation is in-memory, explicitly armed and audited, and exists for demonstrations only.
